POS Cafe — Developer Reference & Change Guide

Version: 0.1.0 · **Generated:** September 2026 **Repository:** `codezen879/POS_CAFE` (GitHub) · **Production:** `https://pos-cafe-eight.vercel.app`

This document gives a developer everything needed to safely modify and deploy the POS Cafe application: architecture, data model, every API route, key UI flows (staff table service, kitchen display, billing, and the mobile QR ordering feature), conventions, and hard-won pitfalls.

---

1. Project Overview

POS Cafe is a restaurant/café **point-of-sale** web application with:

- **Back-office app** (staff): table service, order taking, kitchen display (KDS), billing with GST + service charge, payments (split supported), receipts (printable), customer loyalty + reviews, menu & add-on management, inventory, staff management, settings, tax rates, and reports.
- **Mobile ordering app** (guest): public pages at `/m` and `/m?table=<id>` where guests browse the menu, customize add-ons, add items to a cart, and send orders straight to the kitchen via Table-Service QR codes.

Stack

Layer	Technology
Framework	Next.js **15.2.8** (App Router) on Node
UI	React **19.0.1**, Tailwind CSS 3.4, shadcn-style `components/ui`, lucide-react icons
Auth	NextAuth **5.0.0-beta.25** (Credentials provider, JWT strategy)
ORM	Prisma **5.22** (dual schema — see §4) + `@prisma/adapter-pg` (driver adapters)
State (client)	Zustand (cart)
Forms / misc	react-hook-form, zod, date-fns, recharts, qrcode.react, react-hot-toast
Local dev DB	MySQL (`prisma/schema.prisma`)
Production DB	PostgreSQL on **Supabase** (`prisma/schema.postgres.prisma`)
Hosting	**Vercel** (project `pos-cafe`, account `codezen879`)

> **Why dual databases?** Local development targets MySQL for speed and developer familiarity; the production deployment on Vercel targets a Supabase PostgreSQL instance. Nothing in application code knows which DB it is using — the Prisma client is selected at runtime (see §4).

---

2. Repository Layout

```
POS_CAFE/
├── prisma/
│   ├── schema.prisma            # MySQL schema (local dev)
│   ├── schema.postgres.prisma   # PostgreSQL schema (Supabase/Vercel) — generator outputs to ../src/ge
│   ├── seed.ts                  # MySQL seed
│   └── seed.pg.ts               # PostgreSQL seed (idempotent upserts)
├── public/
├── docs/                        # this document
├── next.config.ts               # minimal (reactStrictMode: true)
├── tsconfig.json                # path alias "@/*" → "./src/*"
├── .env .env.local              # NOT committed; see §5
├── .gitignore                   # also ignores prod-server.log
├── package.json
└── src/
    ├── generated/pg/            # generated Prisma client for PostgreSQL (ignored by git)
    ├── app/
    │   ├── layout.tsx           # root layout: <Providers> + globals.css
    │   ├── globals.css          # Tailwind directives
    │   ├── (auth)/login/page.tsx# PIN / password login form
    │   ├── (app)/layout.tsx     # guarded app shell (redirects to /login)
    │   ├── (app)/...            # dashboard, tables, kitchen, menu, orders, billing,
    │   │                        # customers, inventory, staff, settings, reports
    │   ├── m/page.tsx           # PUBLIC mobile guest menu (the /m app)
    │   └── api/...              # all API route handlers (see §8)
    ├── components/
    │   ├── ui/                  # shadcn-style primitives
    │   ├── app-shell.tsx        # sidebar + topbar for staff app
    │   ├── providers.tsx        # SessionProvider + ThemeProvider + Toaster
    │   ├── pos/                 # table-grid.tsx, table-terminal.tsx, bill-view.tsx,
    │   │                        # receipt.tsx, open-table-dialog.tsx, manage-tables-dialog.tsx
    │   ├── dining/dining-menu.tsx# guest mobile ordering UI
    │   ├── kitchen/kitchen-board.tsx
    │   ├── billing/billing-list.tsx
    │   ├── orders/orders-list.tsx
    │   ├── menu/menu-manager.tsx
    │   ├── inventory/inventory-manager.tsx
    │   ├── staff/staff-manager.tsx
    │   ├── customers/customers-list.tsx
    │   ├── reports/reports.tsx
    │   ├── settings/settings-manager.tsx
    │   └── dashboard/dashboard.tsx
    ├── store/cart.ts            # zustand cart store shared by ordering UIs
    ├── config/nav.ts            # navigation tree with per-role filtering
    └── lib/
        ├── auth.ts              # NextAuth config (credentials, JWT, trustHost)
        ├── session.ts           # requireUser / requireRole / role helpers
        ├── api.ts               # apiAuth(...roles) guard for route handlers
```

```
        ├── prisma.ts            # runtime client selection (pg vs mysql)
        ├── billing.ts           # computeBillForSession() — the tax engine
        ├── serialize.ts         # toPlain() — Decimal/Date safe serialization
        └── utils.ts             # cn, formatCurrency, generateReference, jsonError, …
```

---

3. Runtime & Toolchain

- Node **24.14.1**, npm (Vercel's build environment runs Node 22+).
- Install once: `npm install` (postinstall runs `prisma generate` for **both** schemas — this is why both clients exist).
- Verified dev commands:
- `npm run dev` — local dev server
- `npm run build` — production build (the authoritative check before committing)
- `npm run vercel-build` — the Vercel install/build: generates both Prisma clients then `next build`
- `npm run db:push` / `npm run db:push:pg` — push schema to MySQL / PostgreSQL
- `npm run db:seed` / `npm run db:seed:pg` — seed (Postgres variant requires `POSTGRES_URL`)
- `npm run generate:pg` — regenerate the PostgreSQL client
- **Linting:** there is no ESLint config; `npm run lint` (`next lint`) is effectively a no-op/deprecated. Use `npm run build` as the gate (Next runs its own type checks).

---

4. The Two-Backend Architecture (Important)

4.1 Two schemas, two generated clients

	Local	Production
Schema file	`prisma/schema.prisma`	`prisma/schema.postgres.prisma`
Datasource	`mysql` / `DATABASE_URL`	`postgresql` / `POSTGRES_URL`
Generated client	`@prisma/client` (in `node_modules`)	`../src/generated/pg` (custom output)
Driver	default (mysql2 under the hood)	`@prisma/adapter-pg` (driver adapters)

The PostgreSQL schema adds `previewFeatures = ["driverAdapters"]` and `output = "../src/generated/pg"`.

4.2 Runtime selection — `src/lib/prisma.ts`

```
export const prisma: PrismaClient = process.env.POSTGRES_URL
  ? createPostgresClient()   // Pool + PrismaPg adapter + generated/pg client
  : new PrismaClient();      // default MySQL client
```

- The exported object is typed as the MySQL `PrismaClient` to keep compile-time field types identical; the pg client is cast.

- Selecting by the presence of `POSTGRES_URL` means: set `POSTGRES_URL` locally and the *same code* talks to Supabase — this is the pattern used by the QA scripts and local testing.
- A global cache (`globalThis.prisma`) avoids leaking connections during dev hot-reloads.

4.3 Changing the schema — the golden rule

Every model/field/enum change must be mirrored in BOTH schema files, or the MySQL and PostgreSQL apps drift. Procedure:

1. Edit `prisma/schema.prisma` and `prisma/schema.postgres.prisma` identically.
2. `npm run db:push` (MySQL) and, with `POSTGRES_URL` set, `npm run db:push:pg` (Supabase).
3. `prisma generate` runs automatically on install; or run `npm run generate:pg` for the pg client.
4. Update both seeders if you seeded sample data that should exist in both DBs.

---

5. Environment Variables & Secret Management

`.env` / `.env.local` are gitignored. Required variables:

Variable	Used by	Notes
`DATABASE_URL`	MySQL (schema.prisma)	Local dev only. **Also present (legacy) in Vercel but ignored on the pg path.**
`POSTGRES_URL`	PostgreSQL path + Supabase	**The signal that selects the pg backend.** On Vercel this is the **transaction pooler** URL.
`AUTH_SECRET`	NextAuth v5 JWT signing	Any long random string; set identically across local + Vercel.
`AUTH_URL`	NextAuth	Production URL (`https://pos-cafe-eight.vercel.app`); Vercel sets it automatically but can be forced.
`NEXT_PUBLIC_APP_NAME`	Public branding	e.g. "POS Cafe"
`NEXT_PUBLIC_APP_URL`	Public links	Production URL

Supabase connection URLs (project `nurjnwcxibvpiofgijtf`)

- **Transaction pooler (`:6543`)** — use on Vercel (serverless-safe) and inside transactions.
- **Session pooler (`:5432`)** — use for local scripts, `prisma db push`, seeds.
- The **legacy direct host is IPv6-only** and will hang/never connect — do not use it.

Deploy environment

- Vercel project `pos-cafe` (account `codezen879`). Production + Preview share the same env values.
- Deploy from the **project directory only**: `npx vercel --prod --yes`. Running `vercel` from your home directory triggers the "deploying home directory" prompt and fails.
- `trustHost: true` in `src/lib/auth.ts` is required for NextAuth v5 to trust the Vercel host (otherwise callbacks/sign-ins break).
- **Vercel blocks vulnerable Next versions**: it refused to build Next **15.2.4** ("Vulnerable version of Next.js detected", CVE-2025-66478). The lockfile pins `next@15.2.8`, `react@19.0.1`, `react-dom@19.0.1`. Never downgrade below 15.2.8 without a release, or deploys will be rejected.

Seeded credentials (both DBs)

Role	Email	Password	PIN
SUPER_ADMIN (Store Admin)	admin@poscafe.example	admin123	1234
CASHIER (Rahul)	cashier@poscafe.example	staff123	5678
KITCHEN (Chef Ayesha)	kitchen@poscafe.example	staff123	1111

---

6. Data Model

Enums: `Role` (SUPER_ADMIN, ADMIN, MANAGER, CASHIER, WAITER, KITCHEN) · `TableStatus` (AVAILABLE, OCCUPIED, RESERVED, CLEANING, CLOSED) · `SeatStatus` (FREE, OCCUPIED) · `SessionStatus` (OPEN, CLOSED, CANCELLED) · `OrderType` (DINE_IN, TAKEAWAY, DELIVERY, ONLINE) · `OrderStatus` (DRAFT, SENT_TO_KITCHEN, PREPARING, READY, PARTIALLY_SERVED, SERVED, CANCELLED) · `BillStatus` (DRAFT, ISSUED, PARTIALLY_PAID, PAID, VOID, REFUNDED) · `PaymentMethod` (CASH, CARD, UPI, WALLET, SPLIT, ON_ACCOUNT) · `PaymentStatus` (PENDING, COMPLETED, FAILED, REFUNDED, VOIDED) · `StockMovementType` (PURCHASE, CONSUMPTION, ADJUSTMENT, STOCKTAKE, WASTAGE) · `LoyaltyType` (EARN, REDEEM, EXPIRE, ADJUST).

Model	Table	Responsibilities / key relationships
`User`	users	Staff. `passwordHash` + `pin` (bcrypt). Optional `storeId`. Related to shifts/sessions/orders/payments/bills/invoices.
`Store`	stores	Single store is the norm (`store.findFirst()` everywhere). Holds GSTIN, address, currency, tax rates, settings, sessions, tables.
`StaffShift`	staff_shifts	Clock in/out with cash handling.
`TaxRate`	tax_rates	`code` (CGST/SGST/...), `rate`, unique `[storeId, code]`. Used for GST split.
`Setting`	settings	Key/value store, unique `[storeId, key]`. e.g. `service_charge_percent`, `loyalty_points_per_rupee`.
`MenuCategory`	menu_categories	`slug` unique; `icon`, `sortOrder`, active flag; 1→N Products; owns 1→N AddonOptions (`addons`).
`Product`	products	`basePrice` (Decimal), `costPrice`, `taxRateId`, `isVeg`, `isBestseller`, `isAvailable`, `prepTimeMins`, `maxOrderQty`, `image`, `sortOrder`. N—M with AddonOption via `ProductAddon`.
`AddonOption`	addon_options	Add-on ("Whipped Cream", "Extra Shot") with `price`. Optional `categoryId` FK → MenuCategory (`onDelete: SetNull`): **each add-on belongs to at most one category** — a category is the parent, its add-ons are children, and no two categories can share an add-on.
`ProductAddon`	product_addons	Join: `productId × addonId` with `mandatory`, `maxSelect`, `sortOrder`.
`Floor`	floors	Optional grouping for tables.
`DiningTable`	tables	`tableName` (T1...), `seatCount`, `status`, `qrCode`. Owns `seats`. Has many sessions.
`TableSeat`	table_seats	Per-table seats, unique `[tableId, seatNo]`.
`Customer`	customers	Loyalty: `loyaltyPoints`, `tier`. Has sessions, reviews, loyaltyTxns.
`TableSession`	sessions	**The unit of service.** `sessionNumber` (unique, e.g. `TAB-…`), `tableId`, `guestCount`, `customerId`, `servedById`, `status` (OPEN/CLOSED). Has orders; 1:1 bill.
`Order`	orders	`orderNumber` (unique), `sessionId`, `type`, `status`; lifecycle timestamps `sentToKitchenAt / prepStartedAt / readyAt / servedAt / cancelledAt`.
`OrderItem`	order_items	Snapshot of `name` + `unitPrice` + `quantity` + `note` (price/name snapshotted so later menu edits don't rewrite history). **Item-level pipeline fields**: `status` (`IN_PROCESS` default for new lines, then `READY` / `SERVED` / `DEFECTIVE`), `priority` (waiter fire-mark, blue), `readyAt` / `servedAt` / `defectiveAt` / `defectiveReason`. Indexed `[status, priority]` for the kitchen/waiter feeds.
`OrderItemAddon`	order_item_addons	Addon snapshot per order item: `name`, `price`, `quantity`.
`Bill`	bills	`billNumber` unique, 1:1 with session. Holds computed `subtotal / discountAmount / taxTotal / serviceCharge / roundOff / total / paidAmount / dueAmount`, plus `issuedById`, `voidReason`, tax lines, payments, invoice 1:1, credit note 1:1.
`BillTaxLine`	bill_tax_lines	One row per tax code on the bill (rate, baseAmount, taxAmount).

Model	Table	Responsibilities / key relationships
`Payment`	payments	`method`, `amount`, `status`, `receivedById`, `paidAt`. Split payments = many rows against one bill.
`Invoice`	invoices	GST invoice snapshot (taxableValue, cgst, sgst, igst, total).
`CreditNote`	credit_notes	`noteNumber`, `amount`, `reason`.
`LoyaltyTransaction`	loyalty_transactions	points earn/redeem ledger per customer.
`Review`	reviews	Star rating + comment, optionally tied to customer/session.
`Supplier`	suppliers	Ingredient supplier with contact info, ideologies for movements.
`Ingredient`	ingredients	Stocked raw material: `stockQty`, `reorderLevel`, `costPerUnit`.
`RecipeItem`	recipe_items	How much of an ingredient a product consumes (`qtyUsed`).
`StockMovement`	stock_movements	`type`, `quantity`, `unitCost`, `note`. Ingredient stock audit trail.

Design notes to respect while coding:

- **Money is** `Decimal(10,2)` in the DB. Prisma returns `Decimal` objects that **cannot be serialized to JSON** or passed into client components directly — always wrap payloads with `toPlain()` (see §7).
- **Price/name snapshots**: `OrderItem.name/unitPrice` and `OrderItemAddon.name/price` are copied at order time. Never recompute them from `Product` on read.
- **Order lifecycle statuses** flow DRAFT → SENT_TO_KITCHEN → PREPARING → READY → (PARTIALLY_)SERVED, with `CANCELLED` a terminal state. The kitchen board only shows statuses in that pipeline.
- **Dish-level pipeline (cooking + service)**: `OrderItem` carries its own `status` (`IN_PROCESS` → `READY` → `SERVED`) or `DEFECTIVE`. The order's coarse status is **derived** from its items (`rollupOrderStatus` in `src/app/api/pos/items/[itemId]/route.ts`: all terminal → `SERVED`, some terminal → `PARTIALLY_SERVED`, all active ready → `READY`, any in-progress/ready → `PREPARING`, no items left → `CANCELLED`). `DEFECTIVE` dishes are written off as waste and are **never billed** (§9.5).
- **Session === occupancy**: a table is "occupied" while a session with `status: OPEN` exists. Opening creates the session and sets `DiningTable.status = OCCUPIED`; paying the full bill closes the session and frees the table (see §9).

---

7. Server Data Plumbing & Serialization

7.1 `toPlain()` — `src/lib/serialize.ts`

Every client component receives its data as JSON over the server boundary. Prisma `Decimal` and `Date` fields break that (Next error: "Only plain objects can be passed to Client Components"). The one-liner for that is:

```
export function toPlain<T>(value: T): T {
  return JSON.parse(JSON.stringify(value, (_k, v) => {
    if (isDecimal(v)) return v.toNumber();
    if (v instanceof Date) return v.toISOString();
    if (v instanceof Map) return Object.fromEntries(v);
    if (v instanceof Set) return Array.from(v);
    return v;
  }));
}
```

Pattern: in server components/page.tsx, query with Prisma then return `<ClientComponent data={toPlain(data) as any} />`. See `src/app/m/page.tsx` and `src/app/(app)/tables/page.tsx` for live examples.

7.2 Dynamic rendering

Pages that call `auth()` or `searchParams` are inherently dynamic. The public `/m` page explicitly sets `export const dynamic = "force-dynamic"` because it must reflect live menu + table state. New public pages should do the same; new staff pages mostly inherit dynamism from `auth()`.

7.3 Helpers — `src/lib/utils.ts`

- `cn(...)` — `clsx` + `tailwind-merge` for conditional classes (use everywhere, never template strings).
- `formatCurrency(n, "INR")` — `Intl.NumberFormat("en-IN")`; returns `₹<value>` fallback. **Note: no space** — `"₹30.00"`.
- `formatDate / formatDateTime / formatTime / timeAgo` — en-IN locale helpers. `timeAgo` powers the KDS.
- `generateReference(prefix)` — `PREFIX-<base36 timestamp><rand4>`; used for `ORD-…`, `TAB-…`, `BILL-…`.
- `initials(name)`, `slugify(str)`.
- `jsonError(message, status=500)` — returns `Response.json({ error })`; the caller convention.

---

8. Authentication & Authorization

8.1 NextAuth config — `src/lib/auth.ts`

- **Provider:** Credentials only. The form submits `email` (identifier: email **or** user id, resolved with `findFirst OR`), plus either `pin` or `password`.
- **PIN login** (fast path for terminals): compares against `user.pin` via bcrypt. If `pin` is supplied, PIN wins over password.
- **Password login**: compares against `user.passwordHash`.
- **Strategy:** JWT (`session: { strategy: "jwt" }`). The `jwt` and `session` callbacks stamp `id`, `role`, `storeId` onto the token and session user.
- `pages.signIn = "/login"`, `trustHost: true`.
- Route handlers: `src/app/api/auth/[...nextauth]/route.ts` exports `{ handlers }`.

8.2 Server-side guards

- `src/lib/session.ts`:

- `getCurrentUser()` — `auth()?.user ?? null`.
- `requireUser()` — redirects to `/login`.
- `requireRole(...roles)` — redirects to `/` if role not allowed (empty = any authenticated user).
- `isManager(role)` / `canManage(role)` — `SUPER_ADMIN | ADMIN | MANAGER`.
- `isServerStaff(role)` — role is CASHIER/WAITER/MANAGER/ADMIN/SUPER_ADMIN (usable for terminal access checks).
- `src/lib/api.ts`:
- `apiAuth(...roles)` — for route handlers. Returns the `{ id, role, storeId }` user or a `Response` (401/403). **The universal route-handler pattern:**

```
export async function GET() {
  const user = await apiAuth("SUPER_ADMIN", "ADMIN", "MANAGER");
  if (user instanceof Response) return user;
  // ... do work ...
  return Response.json({ ... });
}
```

- **There is no middleware.ts** — all protection is per-page (layout) and per-route (`apiAuth`). The `(app)` layout redirects unauthenticated users; public paths (`/login`, `/m`, `/api/m/*`, and the endpoints intentionally listed in §8.3) are unprotected by design.

8.3 Public (unauthenticated) endpoints — intentional

Endpoint	Why public
`/login`, `/api/auth/*`	Auth entry point
`/m`, `/m?table=<id>`	Guest mobile menu
`GET /api/m/tables`	Guest checkout lists open tables
`POST /api/m/orders`	Guest places an order (safe because product names/prices are re-hydrated server-side — see §10.3)

8.4 Role gating in the UI

`src/config/nav.ts` defines the sidebar; `getNavGroups(role)` filters items with an `roles` array (Menu, Inventory, Staff, Reports → manager; Settings → SUPER_ADMIN/ADMIN). Add a new page by adding a `NavItem`; add `roles` to hide it from lower roles.

---

9. Staff App — Core Flows

9.1 Login (`/login`)

Client form with a PIN/Password toggle; calls `signIn("credentials", { redirect: false, ... })`, toast on error, then `router.push(callbackUrl)`.

9.2 Table Service (`/tables` → `TableGrid` → `TableTerminal`)

1. **Server query** (`(app)/tables/page.tsx`): loads tables (with floor, OPEN sessions, customer, orders with items+addons), the menu (categories → active products with taxRate + addons), store (with taxRates), and recent customers for quick attach. All payloads run through `toPlain` .
2. `TableGrid` renders colored table cards. Click behavior:

- occupied + has session → open `TableTerminal` ;
- otherwise → `OpenTableDialog` (guest count + optional customer); on success calls `POST /api/pos/tables/<id>/open` → creates session `TAB-…` , frees nothing, sets table OCCUPIED.
- A **Menu QR** dialog shows the generic `/m` QR and one QR per currently open table (value `/m?table=<tableId>`).
- Manager-only **Manage** dialog (PATCH/DELETE via `/api/tables/:id`).

1. `TableTerminal` — the ordering/billing screen for one table:

- Category chips → product cards (veg mark, code, price, prep time). Tapping a product with add-ons opens the add-on stepper (qty per add-on); without add-ons it adds directly.
- Cart panel (zustand `useCart`): line items with add-on lines, qty steppers, remove; **Send to kitchen** → `POST /api/pos/sessions/<id>/order` (staff-authorized), then `toast.success("Order sent to kitchen")` , clears cart, refreshes.
- **Bill** tab → `BillView` (§9.4).

1. **Live order management (Placed orders)** — a panel below the cart lists every active order for the table with `orderNumber` , live status badge, and `timeAgo(placedAt)` :

- **Editable while** `DRAFT` / `SENT_TO_KITCHEN` / `PREPARING` (`EDITABLE_STATUSES` in `table-terminal.tsx`): qty steppers (`PATCH`), per-item remove (`DELETE`), and a **Cancel** button. Edits lock once `READY` , showing "This order can no longer be edited."
- **Cancellation is reason-gated** (see §9.6): the Cancel button (`CANCELLABLE_STATUSES` = `DRAFT/SENT_TO_KITCHEN/PREPARING/READY`) opens a nested dialog instead of `window.confirm` . For `PREPARING` / `READY` orders the **only** reason offered is **"Food defect / damaged"** — confirming calls `PATCH /api/pos/orders/<id>` `{ status: "CANCELLED", reason, note }` , which auto-records a `WasteRecord` at cost price and writes off recipe ingredients (§9.6).
- **Item API** lives in `src/app/api/pos/orders/[id]/items/[itemId]/route.ts` : `PATCH { quantity }` clamps 0–99 and rejects non-editable statuses with 409; `DELETE` removes the item and **auto-cancels the order when the last item is removed**.
- TableGrid keeps the open terminal live: a `useEffect` re-syncs `terminalTable` from the refreshed `tables` prop, so send/edit/cancel/bill updates appear without reopening the table. It **only swaps when the refreshed table still has an open session** — if a settled table briefly refreshes with `sessions: []` it keeps the current terminal rather than blanking the popup (§9.4).

9.3 Kitchen Display (`/kitchen` → `KitchenBoard`)

- **Item-level board** (replaced the per-order kanban): tracks each *dish line*, not the whole order. Polls `GET /api/pos/items` every **8 s** (setInterval + `load()`). That feed flattens the items of every non- `CANCELLED` / `DRAFT` order across all OPEN sessions, with `status` , `priority` , `readyAt` / `servedAt` / `defectiveAt` , table name, orderNumber, orderedBy and add-ons (sessions ordered by `openedAt`).
- Two live columns, sorted **priority first, then FIFO by** `createdAt` :
- **In process** (amber cards) — `status: IN_PROCESS` , each with a **Mark ready** button → `PATCH /api/pos/items/<itemId>` `{ status: "READY" }` .

- **Ready** (pink cards) — `status: READY`, labeled "awaiting waiter" (the waiter serves them).
- Priority items render **blue** with a `PRIORITY` chip (`ItemCard`) and always sort to the top.
- A **Done** strip below shows Served / written-off chips (latest first, dimmed) so served dishes leave the active queue.
- Colors are presentation only (`status` / `priority` are the source of truth); the kitchen never sees served/defective items in the active columns.

9.3a Waiter tab (`/waiter` → `WaiterBoard`)

- New nav entry (HandPlatter icon) in `src/config/nav.ts`; page `src/app/(app)/waiter/page.tsx` (auth-guarded) renders `src/components/waiter/waiter-board.tsx`.
- Polls the same `GET /api/pos/items` every **5 s** (faster than the kitchen so serving keeps up). **Orders are clubbed by table**: one big box per table containing every pending order of that table stacked inside (order headers shown as `orderNumber`, items grouped under their order). Boxes sort priority-first, tables alphabetically, Takeaway last; blue = priority.
- Every **item row** (pending `IN_PROCESS` red / ready `READY` pink status badge) carries its own compact controls:
- **Zap** priority toggle → `PATCH { priority }` (row + order header turn blue, box pins to top).
- **Served** → `PATCH { status: "SERVED" }` — dish leaves the queue and becomes billable.
- **Reject** → dialog (reason `DEFECTIVE_FOOD` / `OTHER` + optional note) → `PATCH { status: "DEFECTIVE", reason, note }` for that single item. `IN_PROCESS` items may still be rejected (a dish being cooked can still be written off if guests complain before it's plated).
- Reject dialog warns the dish "will not appear on the bill". Both boards operate per item and never mutate orders directly. Fully terminal orders are listed in the "Served & written off" strip below.

9.4 Billing & Payment (`BillView`, `Receipt`)

1. `BillView` loads `GET /api/pos/sessions/<id>/bill-detailed` (bill + payments + taxLines + session/table/customer/orders).
2. If no bill yet: discount UI (None / ₹ flat / % off; optional "Redeem points" — see §12) → **Generate bill** → `POST /api/pos/sessions/<id>/bill`. That route reads the `service_charge_percent` Setting, calls `computeBillForSession()`, and **upserts** the Bill keyed on `sessionId`, replacing `taxLines` wholesale (`deleteMany` + `create`). Bill becomes `ISSUED` with `dueAmount = total`.
3. Payment grid (CASH / UPI / CARD / SPLIT) + amount input (defaults to due) → **Receive** → `POST /api/pos/bills/<id>/pay`:

- Creates a `Payment` (amount, method, receivedById).
- Updates `paidAmount` / `dueAmount`; `isPaid` when `paidAmount ≥ total - 0.01` → status `PAID` else `PARTIALLY_PAID`.
- **On full settlement inside a `$transaction`:** closes the session (`CLOSED`, `closedAt`), frees the table (`AVAILABLE`), and if a customer is attached, awards loyalty points (`loyalty_points_per_rupee` Setting, `Math.floor(total)`) + inserts an EARN `LoyaltyTransaction`.

1. Settled bill → `Receipt`: printable 80 mm receipt, plus "This table left a review?" star dialog (`POST /api/reviews`). Printing is **not** `window.print()` inside the dialog — `receipt.tsx` exposes `printReceipt(bill, store)` which renders the same escaped HTML (`buildReceiptHtml`) into a **hidden iframe and prints that**, so the popup never closes/blankes when the browser print dialog opens (`Receipt` renders the identical HTML via `dangerouslySetInnerHTML` for screen preview).

2. **Bill stays visible after settlement.** The popup no longer blanks after payment: `TableGrid`'s live-sync effect (§9.2.4) only swaps its `terminalTable` when the refreshed `/tables` feed still has an *open* session for that table — when a settled table briefly re-appears with `sessions: []` it keeps the current terminal instead of nulling it (older behaviour blanked the dialog right when the change was visible and printing could be lost).

9.4a Viewing & cancelling bills (`/billing` → `BillingList`)

- `BillingList` renders recent bills with status badge + amounts and a per-row **View** button: it fetches `GET /api/bills/<id>` (bill + payments + taxLines + session/table/customer/orders) and shows the full `Receipt` in a dialog — re-print past bills here.
- Unpaid bills (`DRAFT` / `ISSUED` / `PARTIALLY_PAID`) get a **"Cancel this bill"** button (`window.confirm` then `PATCH /api/bills/<id>`): the bill → `VOID` (`voidedById` / `voidedAt` / `voidReason`), the **session stays OPEN and the table stays occupied** so a corrected order/bill can be re-issued. Paid bills show "Paid bills cannot be cancelled." and already voided/refunded bills "Bill already cancelled." — no button.
- `src/app/api/bills/[id]/route.ts` gates **PATCH to managers only**: rejects `PAID` (must refund first) and already `VOID` / `REFUNDED` with 409.

9.5 Billing math — `src/lib/billing.ts` (`computeBillForSession`)

Consumed by the bill route; single source of truth for totals:

```
lineBase     = (unitPrice + Σ addon.price×qty) × qty          // per item
subtotal     = Σ lineBase
taxBucket    = product.taxRate.rate split in two: CGST + SGST, each rate/2
taxAmount    = lineBase × (rate/2) / 100  → per bucket
discount     = if PERCENTAGE: subtotal×value/100
               if FIXED: min(value, subtotal)
taxableBase  = subtotal - discount
taxTotal     = Σ bucket × taxableBase/100 × ... (bucket applies to taxableBase)
serviceCharge= subtotal × serviceChargePercent/100
total        = taxableBase + taxTotal + serviceCharge
```

All steps `round2` (×100/100). Excludes orders with status `CANCELLED` or `DRAFT`, **and any dish whose `OrderItem.status === "DEFECTIVE"`** (written-off dishes are skipped via `continue` before the line is summed, so subtotal/tax/service all exclude them). **When you change pricing logic, change this file — never inline the math in route handlers.**

The coexisting `GET /api/pos/sessions/:id/bill-detailed` route applies the same filter to its line items (`status NOT IN ["DEFECTIVE"]`), so the bill UI and math agree.

9.6 Waste & trash management (`/waste` → `WasteList`)

Tracks inventory used with **no revenue** — defective returns, cancels, spillage, expiry — valued at **cost price**, and writes those ingredients off from stock automatically.

- **Cancel policy** (`src/app/api/pos/orders/[id]/route.ts`): a `CANCELLED` patch requires a reason once prep has started.
- `DRAFT` / `SENT_TO_KITCHEN` → free cancel, any of `DEFECTIVE_FOOD` / `NOT_STARTED` / `OTHER` (no waste).

- `PREPARING` / `READY` → `DEFECTIVE_FOOD` **only**; any other reason → **409** ("the guest must pay"). This is why the Table Service dialog offers a single defect option once the kitchen started (§9.2.4).
- `PARTIALLY_SERVED` / `SERVED` → must be billed and paid; cancelled orders are final, `SERVED` is final.
- **Auto-waste**: `recordWasteFromOrder()` wraps the cancel in a `$transaction` — creates a `WasteRecord` (`source ORDER_CANCEL`, `reason DEFECTIVE_FOOD`, captures `tableName` + `note`), snapshots each `OrderItem` into a `WasteItem` at `Product.costPrice` (fallback `unitPrice`), totals `totalCost`, then for every recipe ingredient (`order item → product.recipe`) creates a `WASTAGE` `StockMovement` (`qty = qtyUsed × item.quantity`, clamped to available) and **decrements** `Ingredient.stockQty` by the same value so stock reflects what was thrown away.
- **Item-level defects** (the waiter board's "Defect"): `PATCH /api/pos/items/<itemId>` `{ status: "DEFECTIVE" }` records the same thing for a single dish via `recordWasteForItem()` in the item route — a `WasteRecord` with `source ITEM_DEFECT`, a `WasteItem` linked back by `orderItemId`, recipe-based stock write-offs, all inside the item-update transaction (a double-defect can never create duplicate waste; the guarded `updateMany` returns 409 for the loser).
- **Manual records** (`src/app/api/waste/route.ts`): `GET` lists records (filters `reason` / `source` / `from` / `to` / `limit`, includes items + movements + `recordedBy` + order); `POST` (manager+) creates a record for any `WasteReason` with free-form wasted items + ingredient deductions. `POST /api/waste/:id/ingredients` (manager+) adds more ingredient write-offs to an existing record.
- Schema: `WasteRecord` ↔ `{items: WasteItem[], movements: StockMovement[]}`, `StockMovement.wasteRecordId` links each write-off back to its record; `enum WasteReason { DEFECTIVE_FOOD, NOT_STARTED, SPILLAGE, EXPIRED, OTHER }`. `WasteList` shows today/total value + units written off, and gates the manual dialogs to `SUPER_ADMIN/ADMIN/MANAGER`.

---

10. Mobile QR Ordering (`/m` — the guest feature)

10.1 Route wiring

Public guest ordering lives under three pieces:

Piece	Responsibility
`src/app/m/page.tsx`	`force-dynamic` RSC. Queries store, active categories (products + addons), and if `?table=` present, resolves that table + its latest OPEN session. Passes `table` prop to the client component.
`src/components/dining/dining-menu.tsx`	All guest UX (menu grid, Customize sheet, cart sheet, table chooser, send, success banner).
`GET /api/m/tables`	Public; returns `{ tables: [{ id, tableName }] }` for tables with an OPEN session (drives the checkout "Choose your table" step).
`POST /api/m/orders`	Public; validates and creates the kitchen order (see §10.3).

10.2 UX state machine (`dining-menu.tsx`)

`const tableLocked = !!table?.id && !table?.sessions?.[0];` — a **bound-but-not-open** table (server hasn't opened the session) blocks ordering and shows "This table isn't open yet — please ask your server." Otherwise guests can always add items:

- **Item card = a full** `<button>`; `onClick` opens the Customize sheet when the product has active add-ons, else adds it directly. `active:scale-[0.98]` gives tap feedback (this replaced the earlier silently-disabled buttons — see §14).
- **Customize sheet**: add-on rows are `<button aria-pressed>` toggles; selected rows highlight, show a `<Check>` icon and `+₹<price>`. Quantity stepper (1–99). "Add N to your order" → `addItem` with chosen add-ons.
- **Cart sheet** (bottom sheet, `max-h-[85vh]`): line items with add-on line, qty steppers, remove; subtotal via `computeSummary(lines)` (tax = 0 client-side; real tax is applied server-side at billing only).
- If no table is bound (`!table?.id`), a **table picker** lists open tables from `/api/m/tables`; selecting one enables Send. Send button label switches to **"Pick your table, then send"** until chosen.
- Send → `POST /api/m/orders` with `{ tableId: targetTableId, items: [{ productId, quantity, note, addons: [{id, quantity}] }] }`.
- **Success banner**: `"N item(s) sent to the kitchen for Table T2"` (emerald pill), cart cleared.

Single-flight rule with `targetTableId`: `table?.id` (from the QR) **or** the guest-picked `guestTableId`. Never both.

10.3 Order tamper-resistance (`POST /api/m/orders`)

The guest sends only **IDs** (productId, addon ids + quantities, note) — never names or prices:

1. `tableId` must exist and have an OPEN session, else 404/400.
2. Products are re-fetched by id server-side; only `ProductAddon` links pointing at `isActive` AddonOptions are honored (unknown → filtered out).
3. `name`, `unitPrice = product.basePrice`, and addon `name/price` are **hydrated from the DB**, so a malicious client cannot undercut prices or spoof names.
4. Clamps: `MAX_ITEMS = 50`, `MAX_QTY = 99`, per-addon qty ≤ 9.
5. Order is created with `status: "SENT_TO_KITCHEN"`, `sentToKitchenAt: now`, `orderNumber = ORD-…` (generated on the client terminal path too, via `generateReference`), and appears on the KDS automatically.

Known constraint: guest orders are tied to the table's OPEN session; if the session closes mid-cart the send will 400 ("This table is not open yet").

---

11. Cart Store (`src/store/cart.ts`)

Zustand store shared by `TableTerminal` and `DiningMenu`:

- `CartLine = { key, productId, name, unitPrice, quantity, note?, addons: CartAddon[] }`; `key` **is generated per add** (`<productId>-<Date.now()>-<rand4>`).
- `addItem` **merges** consecutive identical plain (no-addon) lines: same `productId + note` with both addon arrays empty → quantities added. Any add-ons always produce a fresh line (merging addon lines is deliberately not implemented).
- `updateQty` with qty ≤ 0 removes the line.
- `computeSummary(lines)` returns `{ subtotal, tax: 0, total: subtotal }` — used only for the guest cart display; authoritative totals happen in billing.

---

12. Loyalty, Reviews & Customers

- **Attach customer** at table-open (OpenTableDialog selects from `/api/customers`, recent 50) or via phone in the Redeem dialog.
- `GET /api/customers/find?phone=+91…` → customer (or `{ customer: null }`).
- **Redeem** (`bill-view.tsx` → `/api/customers/<id>/redeem`, `POST { points }`) → `{ cashValue }`; 100 points = ₹1, applied as a **FIXED** discount before bill generation. `LoyaltyTransaction` rows track EARN/REDEEM.
- **Earn** happens automatically on full payment for the attached customer (see §9.4).
- **Reviews** `POST /api/reviews { customerId, rating, comment }` from the post-settlement review dialog.

---

13. Menu & Inventory Management

- `MenuManager` (manager-only `/menu`) manages categories/products/add-ons against `GET/POST /api/menu/categories` and `/api/products` + `/api/products/<id>` (PATCH supports `addonIds` replace — deletes then recreates `ProductAddon` rows). **`/api/products` intentionally has no GET handler** → a bare `GET /api/products` returns **405**; that is by design, not a bug.
- Add-ons are `AddonOptions` managed via `GET/POST /api/addons` + `PATCH/DELETE /api/addons/:id` (delete is 409-guarded once referenced by past `order_item_addons`). Ownership is **parent → child**: each add-on carries an optional `categoryId` FK to a `MenuCategory`, so every add-on belongs to exactly one category (or none) and categories can never share add-ons. An add-on is **limited to the category it belongs to**: category add-ons show up only for products in that category, product-level `addonIds` pointing at an add-on owned by a *different* category are **rejected with 400** on both `POST /api/products` and `PATCH /api/products/:id`, and `mergeProductAddons` drops any such link so a mis-linked add-on can never leak into another category. Unassigned add-ons (`categoryId` null) can be attached to any product. PATCHing a category with `addonIds` **moves** ownership of those add-ons to that category (`updateMany` sets `categoryId`) and detaches to `null` any add-ons the category used to own that aren't in the list. The **effective** set a guest/server picker shows is `mergeProductAddons()` (`src/lib/addons.ts`): product links first, then the owning category's `isActive` add-ons, deduped by addon id (product-level wins). `/api/m/orders` validates against that merged set so inherited category add-ons are never silently dropped. `/tables`, `/m`, and kitchen/order flows all consume the merged set; `MenuManager` shows the category owner on add-ons (chips + list rows, "unassigned" when `categoryId` is null), the category edit dialog lets you assign/move add-ons into that category, and the product dialog only offers add-ons that belong to the product's category (or are unassigned).
- `InventoryManager` (manager-only `/inventory`) manages `GET/POST /api/stock` (ingredients, stock movements), plus suppliers via related routes. Order placement does **not** auto-consume stock — but defect-food cancellations and manual `/waste` records **do** write ingredients off as `WASTAGE` movements (§9.6).
- `SettingsManager` (SUPER_ADMIN/ADMIN) edits `/api/settings` (key/value, e.g. `service_charge_percent`, `loyalty_points_per_rupee`). `TaxRates` editable via `/api/tax-rates`.

---

14. API Route Inventory

Conventions: helpers from `@/lib/api` (`apiAuth`), `@/lib/utils` (`jsonError`, `generateReference`). `params` is a `Promise` in Next 15 — always `await params`. Route handlers return `Response.json({...})`; errors are `{ error }` with `jsonError`.

Method	Route	Auth	Purpose
POST	`/api/auth/[...nextauth]`	public	NextAuth handlers
GET	`/api/tables`	manager	All tables + floor + `_count.sessions`
POST	`/api/tables`	manager	Create table (also creates `seatCount` `TableSeat` s)
GET	`/api/tables/floors`	manager	(floors for selects)
PATCH	`/api/tables/:id`	manager	Update name/floor/seats/status; site-aware seat add/remove
DELETE	`/api/tables/:id`	manager	Delete table (blocked while an OPEN session exists)
GET	`/api/floors`, PATCH/DELETE `/api/floors/:id`	manager	Floor CRUD
GET	`/api/menu/categories`	manager/client	Categories (+ products via include)
GET	`/api/products`	—	**No handler (405 by design)**
POST	`/api/products`	manager	Create product
PATCH/DELETE	`/api/products/:id`	manager	Update product (incl. `addonIds` replace; cross-category add-ons rejected 400) / delete
GET	`/api/addons`	manager	List all add-ons (owning `category: { id, name }`, `_count.products` usage)
POST	`/api/addons`	manager	Create add-on `{ name, flavour?, price=0, isActive?, categoryId? }` (category must exist when given)
PATCH/DELETE	`/api/addons/:id`	manager	Update name/flavour/price/isActive/ `categoryId` (null moves add-on back to unassigned; see below re: an already-assigned add-on) / delete (409 once used by past `order_item_addons`)
PATCH/DELETE	`/api/menu/categories/:id`	manager	Update name/icon/isActive + `addonIds` **moves ownership** of those add-ons to this category (detaches any this category owned but that weren't listed) / delete (409 while products exist)
GET	`/api/staff`	manager	List staff (id/name/email/role/isActive)
POST	`/api/staff`	manager	Add staff (bcrypt password + generated 4-digit PIN)
GET	`/api/customers`	staff	Customer list
GET	`/api/customers/find`	staff	`?phone=` lookup
POST	`/api/customers/:id/redeem`	staff	Redeem `{ points }` → `{ cashValue }`
POST	`/api/reviews`	staff	Create review
GET	`/api/tax-rates`	manager	Tax rates

Method	Route	Auth	Purpose
GET	`/api/settings`	admin+	Settings
GET/POST	`/api/stock`	manager	Inventory + stock movements
POST	`/api/pos/tables/:id/open`	staff	Open session (409 if occupied)
POST	`/api/pos/sessions/:id/order`	staff	Staff order → SENT_TO_KITCHEN (server hydrates product identity)
GET	`/api/pos/sessions/:id/bill-detailed`	staff	Bill detail + payments + taxLines + session tree
POST	`/api/pos/sessions/:id/bill`	staff	Generate/refresh bill (upsert on `sessionId`)
GET	`/api/pos/kitchen/orders`	staff	KDS feed (4 active statuses)
PATCH	`/api/pos/orders/:id`	staff	Advance status; stamps lifecycle timestamps; **CANCELLED is reason-gated** (defect-only once PREPARING/READY, else 409) and auto-records waste (`.waste` in response)
PATCH	`/api/pos/orders/:id/items/:itemId`	staff	Update item qty (clamped 1–99); 409 unless DRAFT/SENT_TO_KITCHEN/PREPARING
DELETE	`/api/pos/orders/:id/items/:itemId`	staff	Remove item; removing the last item cancels the order
GET	`/api/pos/items`	staff	**Item-level kitchen/waiter feed**: flattened dish lines of all open-session, non-cancelled/draft orders (status, priority, table, order number, add-ons)
PATCH	`/api/pos/items/:itemId`	staff	Item transition (`IN_PROCESS` ↔ `READY`, → `SERVED`, → `DEFECTIVE`) + `priority` toggle; approved statuses only; guarded `updateMany` (race-safe, 409 on concurrent double-apply); idempotent replays; `DEFECTIVE` writes off waste (`ITEM_DEFECT`) and rolls the order status up
POST	`/api/pos/bills/:id/pay`	staff	Record payment; settles → closes session + frees table + loyalty
GET	`/api/bills/:id`	staff	Full bill detail (payments, taxLines, session/table/customer/orders) for View dialogs
PATCH	`/api/bills/:id`	manager	Cancel unpaid bill → `VOID` (409 if PAID/VOID/REFUNDED); session stays open
GET	`/api/waste`	staff	Waste ledger (filters `reason` / `source` / `from` / `to` / `limit`; items + movements + recordedBy + order)
POST	`/api/waste`	manager	Manual waste record (items + ingredient write-offs)

Method	Route	Auth	Purpose
POST	`/api/waste/:id/ingredients`	manager	Add ingredient `WASTAGE` write-off to an existing record
GET	`/api/m/tables`	**public**	Open tables for guest checkout
POST	`/api/m/orders`	**public**	Guest order (tamper-safe, §10.3)

15. Conventions & Recipes for Common Changes

15.1 Add a new API route

1. Create `src/app/api/<name>/route.ts` (or `src/app/api/<name>/[id]/route.ts`).
2. Use `apiAuth(...roles)` unless it is deliberately public (like the `/api/m/*` endpoints).
3. Guarded by `params: Promise<...>` → `const { id } = await params;`.
4. `req.json().catch(() ⇒ ({}))` for body parsing; validate required fields; respond with `jsonError(msg, code)` on failure.
5. Wrap DB work in `try/catch` returning `jsonError(e.message)`.

15.2 Add a new page (staff)

1. Create `src/app/(app)/<name>/page.tsx` — server component: `auth()` guard (the `(app)` layout already redirects, but pages that branch on role still call `auth()`), Prisma queries, `toPlain()` the payloads, render a `"use client"` component from `src/components/`.
2. Register navigation in `src/config/nav.ts` (add `roles` to gate by role).
3. If role-conditional UI is needed server-side, use `requireRole(...)` from `@/lib/session`.

15.3 Add a public/page-less path (guest)

- Server page under `src/app/<route>/page.tsx` with `export const dynamic = "force-dynamic"`.
- Guest data endpoints under `src/app/api/m/*` and mark them explicitly public in §8.3 and §14.

15.4 Add/extend a database field

Follow §4.3 exactly (both schemas → push both → generate → seed both). Then: if it's a menu/UI-visible field, update the matching `toPlain()` consumer. **Do not forget to run `npm run generate:pg` locally or the pg import will fail to typecheck** (`vercel-build` does this automatically).

15.5 Styling

- Shadcn-style primitives in `src/components/ui/*` (Button, Dialog, Input, Label, Select, Switch, Tabs, Badge, Card, Textarea, Toaster). Use those instead of hand-rolled DOM.
- Variant classes via `cn("bg-…", cond && "…")` and inline `Record<status, class>` maps (see `statusStyle` in `table-grid.tsx`, `COLUMNS` in `kitchen-board.tsx`).
- Paper-like receipts, bottom sheets and the KDS use plain Tailwind with arbitrary values (`max-h-[85vh]`, `pb-[max(0.75rem,env(safe-area-inset-bottom))]`) — keep the safe-area padding on mobile.

15.6 Verifying changes

1. `npm run build` (does type checking).
2. Local: `npm run dev`, log in, exercise the touched flow.
3. Against production DB locally: set `POSTGRES_URL` (session pooler, `:5432`) and run the same app (the QA harnesses in §16 do this through the real origin).
4. Deploy: `npx vercel --prod --yes` **from the project directory**.

---

16. QA & Verification Harness

During the mobile-ordering work, browser QA was driven with `puppeteer-core` + the installed Chrome (`C:\Program Files\Google\Chrome\Application\chrome.exe`) from a scratch dir, against `$env:BASE` (`https://pos-cafe-eight.vercel.app`). Scripts live outside the repo in a temp workspace and use fetch-cookie login + real open-table flows; a `cleanup_t2.mjs` helper resets test table `T2` (closes OPEN sessions, sets AVAILABLE). Observations from that work:

- Always clean the test table between runs (an OPEN leftover session makes re-open return 409).
- Cold-start latency matters: wait for selectors rather than fixed sleeps; allow a few seconds after first request.
- Assert via **accessible state** (`aria-pressed`, rendered text, `disabled`) rather than pixel heuristics.
- `body.textContent` **includes inline** `<script>` **text** (Next embeds big bootstrap/flight payloads) — a guest-send assertion waiting on `textContent.includes("sent to the kitchen")` matched a transient script frame even when the send never fired. Wait on the real network response (`page.waitForResponse` for the POST 2xx) and/or use `body.innerText` (rendered text only) for on-screen asserts.
- Newer harnesses exercise the item pipeline end-to-end via `PATCH /api/pos/items/:itemId` and the `/api/pos/items` feed (priority sort, READY/SERVED/DEFECTIVE transitions, idempotent replays, duplicate-waste guard, defective excluded from bills, waste rows linked).

---

17. Pitfalls & Lessons Learned (read before changing anything)

1. `POSTGRES_URL` **presence switches DB backends.** Accidentally setting it locally changes everything; conversely, forgetting it on Vercel yields MySQL's client with no MySQL host → boot errors. It is the single most important env var.
2. **Never deploy without regenerating the pg client**: any schema drift breaks `src/generated/pg` typecheck in Vercel builds.
3. **Prisma relation** `where` **on to-one includes is disallowed.** You can filter `10` filters in the *include* only for to-many relations. Ordering logic that needs conditional to-one data must `findMany` then filter in JS (this shaped `/api/m/orders` addon handling).
4. **Decimal/Date must pass through** `toPlain` before crossing the RSC→client boundary. Missing it throws a hard Next serialization error.
5. **Vercel blocks Next < 15.2.8** (CVE-2025-66478). Never downgrade.
6. `vercel` **fights back if run from the wrong cwd** (home directory → "deploying home directory"). Always `npx vercel --prod --yes` inside `POS_CAFE`.

7. **Supabase legacy direct host is IPv6-only** → use poolers: `:6543` (transaction) on Vercel, `:5432` (session) for scripts/push/seed.
8. `/api/products` **GET → 405 is intentional.** Callers must use the category endpoint.
9. **Money**: `₹30.00` has no space after `₹`; string-matching UIs (e.g. add-on "+₹" chips) depend on this.
10. **Mobile UX regression guard**: item cards are full-tap `<button>`s; never reintroduce separately-disabled inner buttons without visible feedback, or "tap does nothing" returns (the exact bug that drove Round 2 of the mobile fix).
11. **Status lifecycle**: loans on `PATCH /api/pos/orders/:id` — CANCELLED and SERVED are final; transition validation lives in the route, keep it there. The item-level `PATCH /api/pos/items/:itemId` is the same idea per dish: allow-list statuses, guard with conditional `updateMany`, keep transitions in the route, and mirror them if the pipeline ever expands.
12. **Next 15 route params are promises** — `const { itemId } = await params;`. A forget-`await` (or destructuring the wrong key) yields `undefined` IDs and opaque 500s; the item route folder is `[itemId]`, never `[id]`.
13. `document.body.textContent` **leaks inline script text** — for browser-test assertions use rendered text (`innerText`) or network events, not `textContent` (see §16).
14. **Session/table consistency**: session OPEN ↔ table OCCUPIED must always be updated together (create/open in a single request; settle path uses a `$transaction`).

---

18. Deployment & Operations Quick Reference

```
# Install & generate both clients
npm install

# Sync schema to both DBs (set POSTGRES_URL for the pg push)
npm run db:push
npm run db:push:pg

# Seed both (POSTGRES_URL required for pg)
npm run db:seed
npm run db:seed:pg

# Local
npm run dev

# Build check
npm run build

# Deploy production (must run from POS_CAFE dir)
npx vercel --prod --yes
```

Deployment URL: `https://pos-cafe-eight.vercel.app` (Vercel project `pos-cafe`). Verify a new deploy with the login flow (admin@poscafe.example / admin123), the Tables→Menu QR→table chip→Scan path, and the kitchen board.